

Snake Game

Muradxan Ahmadov

Farhad Rahimzade

Shahlar Jafarov

Said Idragimov

1. Introduction

This is a real-time Snake game written in the C programming language. The game has no defined end; the goal is simply to control the snake and collect apples. The game ends only when the snake's head touches either its own body or a wall.

2. Features

- Controlled using W, A, S, D keys
- The snake grows longer as it eats apples
- Apples appear randomly in different parts of the game area
- The game ends when the snake collides with a wall or its own body
- Live score display

3. Initial Problems and Their Solutions

1) No real-time movement:

In the first version of the code, the player had to press Enter after each move. This reduced both the realism and speed of the game. To fix this, we used the `termios` library to switch the terminal into 'raw mode'.

2) Incorrect movement direction:

For example, when the player pressed the 'W' key to go up, the snake would move to the right instead. This

was due to incorrect values set for `dir_x` and `dir_y`. We solved this by assigning the directions logically and correctly.

3) Incorrect apple coordinates:

Sometimes, the apple would appear either on the snake's body or outside the game area, causing major gameplay issues. To solve this, we ensured the apple's coordinates are generated within a valid range using `rand() % (ROW - 2) + 1` and we check for conflicts with the snake's body before placing it.

4. Main Components of the Game

- Grid Generation: A 2D array with borders
- Snake Movement: Updates the coordinates of the snake's head and body
- Apple Mechanics: Random but checked placement
- Collision Detection: Ends the game on hitting walls or the body
- Input Handling: Uses non-blocking input in raw terminal mode

5. Conclusion

This Snake game shows how to build a real-time terminal-based application using C. Writing the code and solving the issues we encountered provided us with valuable experience. After fixing all the bugs and problems, the game became more interesting, functional, and interactive.

```
#include <stdio.h>    // for printf
#include <stdlib.h>    // for rand()
#include <time.h>      // for time()
#include <unistd.h>    // for usleep()
#include <termios.h>   // for terminal settings
#include <fcntl.h>     // for file operations
```

Includes essential libraries for game logic, random number generation, and terminal control.

```
#define ROW 32
```

```
#define COLUMN 62
```

Defines the size of the game field.

```
int user_point = 0;
int length_snake = 1;
int snake_points[ROW][2] = {{16, 5}};
int apple_point[2] = {16, 10};
char snake[ROW] = {'0'};
char grid[ROW][COLUMN];
```

Initial setup for the snake's position, apple position, and grid.

```
void reset_terminal_mode() {...}
void set_conio_terminal_mode() {...}
```

These functions switch the terminal to raw mode for real-time input, and restore it afterward.

```
int kbhit() {...}
int getch() {...}
```

Implements non-blocking keyboard input.

```
void GridGenerating() {...}
```

Draws the borders, snake, and apple onto the grid.

```
void displayGrid() {...}
```

Prints the grid and the score to the terminal.

```
int gameover() {...}
```

Checks if the snake hits a wall or its own body.

```
void grown() {...}
```

Handles the apple-eating logic and increases snake length.

```
void move_snake() {...}
```

Moves the snake forward and updates its body.

```
void process_input() {...}
```

Reads player input and sets snake direction.

```
int main() {...}
```

Main function that runs the game loop.